UNIVERSITÀ DEGLI STUDI DI PADOVA
CORSO DI LAUREA IN INFORMATICA (L-31)

Corso di Ingegneria del Software
Anno Accademico 2025/2026

Specifica Tecnica

Gruppo: NightPRO

Sistema: SmartOrder

swe.nightpro@gmail.com

Data: 2026-03-26
Versione: 0.2

Tabella delle Versioni

Versione	Data	Autore/i	Descrizione delle Modifiche	Verificatore
0.2	2026-03-26	Davide Biasuzzi	Inserimento paragrafo su esecuzione e su tecnologie utilizzate	Leonardo Bilato
0.1	2026-03-20	Davide Biasuzzi	Stesura base documento (Introduzione e Requisiti di Sistema)	Leonardo Bilato

Indice

Componenti del Gruppo 5

1 Introduzione 6
1.1 Scopo del documento . . . 6
1.2 Scopo del prodotto . . . 6
1.3 Glossario . . . 6
1.4 Riferimenti . . . 6
1.4.1 Riferimenti normativi . . . 6
1.4.2 Riferimenti informativi . . . 6

2 Requisiti di sistema 7
2.1 Requisiti software . . . 7
2.2 Requisiti hardware . . . 7

3 Esecuzione del software 8
3.1 Variabili d'ambiente . . . 8
3.2 Avvio manuale (senza Docker) . . . 8
3.2.1 Terminale 1 - Backend (FastAPI) . . . 8
3.2.2 Terminale 2 - Frontend (Next.js) . . . 9
3.2.3 Accesso alla webapp . . . 9
3.3 Deployment in produzione (Railway) . . . 9

4 Tecnologie utilizzate 10
4.1 Linguaggi di programmazione . . . 10
4.1.1 TypeScript . . . 10
4.1.2 Python . . . 10
4.1.3 SQL . . . 10
4.2 Framework e librerie . . . 10
4.2.1 Next.js . . . 10
4.2.2 React . . . 11
4.2.3 Tailwind CSS . . . 11
4.2.4 FastAPI . . . 11
4.2.5 Pydantic . . . 11
4.2.6 OpenAI SDK . . . 11
4.3 Strumenti di persistenza . . . 11
4.3.1 PostgreSQL . . . 11
4.3.2 psycopg2 . . . 12
4.4 Intelligenza artificiale . . . 12
4.4.1 GPT-4o . . . 12
4.4.2 GPT-4o-mini . . . 12
4.4.3 Whisper . . . 12
4.5 Infrastruttura e DevOps . . . 12
4.5.1 Docker . . . 12
4.5.2 Node.js . . . 12
4.5.3 Uvicorn . . . 13
4.5.4 GitHub Actions . . . 13
4.5.5 ESLint . . . 13

Elenco delle figure

Elenco delle tabelle

1 Componenti del gruppo NightPRO. 5
2 Requisiti software per il deployment. 7
3 Requisiti hardware minimi. 7
4 Variabili d'ambiente richieste. 8

Componenti del Gruppo

Cognome	**Nome**	**Matricola**
Biasuzzi	Davide	2111000
Bilato	Leonardo	2071084
Zanella	Francesco	2116442
Romascu	Mihaela-Mariana	2079726
Ogniben	Michele	2042325
Perozzo	Samuele	2110989
Ponso	Giovanni	2000558

Tabella 1: Componenti del gruppo NightPRO.

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere l'architettura software del sistema *SmartOrder*, definendo le scelte tecnologiche, i pattern architetturali e progettuali adottati, e l'architettura di dettaglio di ciascun componente. La Specifica Tecnica costituisce il riferimento principale per la fase di codifica, garantendo coerenza tra progettazione e implementazione.

1.2 Scopo del prodotto

SmartOrder è una piattaforma B2B per l'automazione della gestione degli ordini commerciali. Il sistema consente ai clienti di formulare ordini in linguaggio naturale (testo, audio, immagini) e li converte automaticamente in ordini strutturati pronti per l'integrazione con sistemi ERP aziendali, riducendo errori e tempi di inserimento manuale.

1.3 Glossario

Per evitare ambiguità, i termini tecnici e di dominio utilizzati nel documento sono definiti nel *Glossario v2.0*. I termini presenti nel glossario sono contrassegnati con il simbolo $_{\text{G}}$ alla prima occorrenza in ogni sezione.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme di Progetto v2.0 (Ultima modifica 2026-03-20)
- Analisi dei Requisiti v1.1 (Ultima modifica 2026-03-20)

1.4.2 Riferimenti informativi

- Glossario v2.0 (Ultima modifica 2026-03-20)
- Piano di Qualifica v1.0 (Ultima modifica 2026-03-20)
- Next.js Documentation (Ultima modifica 2026-03-20)
- FastAPI Documentation (Ultima modifica 2026-03-20)
- OpenAI API Documentation (Ultima modifica 2026-03-20)
- PostgreSQL Documentation (Ultima modifica 2026-03-20)
- Analisi dei requisitiG, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Software Architecture Patterns, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Dependency Injection, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Creazionali, SWE 2025-2026 (Ultima modifica 2026-03-26)
- Design Pattern Strutturali, SWE 2025-2026 (Ultima modifica 2026-03-26)

2 Requisiti di sistema

2.1 Requisiti software

Software	**Descrizione**	**Versione minima**
Node.js	Runtime JavaScript per l'esecuzione dell'applicazione	$\geq$ 20.0
npm	Gestore pacchetti Node.js	$\geq$ 10.0
Python	Interprete Python per il modulo AI	$\geq$ 3.11
pip	Gestore pacchetti Python	$\geq$ 23.0
PostgreSQL	Database relazionale	$\geq$ 16
Browser web	Chrome, Firefox, Safari, Edge	Ultime 2 versioni

Tabella 2: Requisiti software per il deployment.

2.2 Requisiti hardware

Componente	**Requisito**
CPU	Processore 64-bit, $\geq$ 2 core
RAM	$\geq$ 4 GB
Spazio disco	$\geq$ 5 GB
Connessione di rete	Richiesta per accesso alle API OpenAI

Tabella 3: Requisiti hardware minimi.

3 Esecuzione del software

In questa sezione viene descritta la modalità di avvio manuale del sistema *SmartOrder* in ambiente di sviluppo.

3.1 Variabili d'ambiente

Prima dell'avvio è necessario configurare un file `.env` nella directory `backend` con le seguenti variabili:

Variabile	Descrizione
`DB_HOST`	Host del database PostgreSQL
`DB_PORT`	Porta del database PostgreSQL
`DB_USER`	Utente del database
`DB_PASSWORD`	Password del database
`DB_NAME`	Nome del database
`HOST`	Host di bind del backend FastAPI
`PORT`	Porta del backend FastAPI
`API_DEBUG`	Abilitazione modalità debug del backend
`OPENAI_API_KEY`	Chiave API per i servizi OpenAI

Tabella 4: Variabili d'ambiente richieste.

3.2 Avvio manuale (senza Docker)

La procedura di avvio descritta in questa sottosezione ha lo scopo di consentire l'esecuzione locale del sistema per attività di verifica tecnica, debug e dimostrazione. Essa non rappresenta la procedura di rilascio in ambiente di produzione, che è invece gestita tramite pipeline di deploy automatizzato sulla piattaforma Railway.

Questa prevede l'utilizzo di due terminali separati, uno per il backend e uno per il frontend.

3.2.1 Terminale 1 - Backend (FastAPI)

1. Posizionarsi nella cartella del backend:

```
cd backend
```

2. Creare l'ambiente virtuale Python:

```
python3.11 -m venv venv
```

3. Attivare l'ambiente virtuale:

```
source venv/bin/activate
# Windows: venv\Scripts\activate
```

4. Installare le dipendenze:

```
pip install -r requirements.txt
```

5. Avviare il backend:

```
python main.py
```

Lasciare questo terminale in esecuzione e aprirne un secondo per il frontend.

3.2.2 Terminale 2 - Frontend (Next.js)

1. Installare le dipendenze del frontend:

```
npm install
```

2. Avviare il server di sviluppo:

```
npm run dev
```

Anche questo terminale deve rimanere in esecuzione durante l'utilizzo dell'applicazione.

3.2.3 Accesso alla webapp

La webapp è disponibile all'indirizzo `http://localhost:3000`. In ambiente di sviluppo, le modifiche al codice vengono applicate in tempo reale tramite aggiornamento live.

3.3 Deployment in produzione (Railway)

Il deployment in produzione del sistema *SmartOrder* è gestito tramite la piattaforma **Railway**, utilizzata come ambiente di hosting per l'applicazione web e per il database PostgreSQL.

In sintesi, il flusso di rilascio è il seguente:

- il codice sorgente viene mantenuto sul repository GitHub del progetto;
- Railway è configurato per monitorare il branch principale del repository (ad esempio `main`) e avviare automaticamente una build ad ogni push;
- durante la build, Railway installa le dipendenze necessarie (pacchetti Node.js e Python) e costruisce l'immagine eseguibile dell'applicazione;
- al termine della build, Railway attiva i servizi applicativi esponendoli tramite URL pubblico HTTPS;
- le variabili d'ambiente di produzione (database, chiavi API, parametri di configurazione) vengono gestite all'interno della console di Railway e non sono versionate nel repository.

In ambiente di produzione:

- il backend (modulo FastAPI) e il frontend (Next.js) vengono eseguiti come servizi gestiti da Railway, rispettando le risorse e i limiti configurati sul progetto;
- il database PostgreSQL è ospitato su un'istanza Railway dedicata, raggiungibile tramite i parametri configurati nelle variabili d'ambiente;
- la piattaforma gestisce automaticamente aspetti infrastrutturali come il bilanciamento del carico, la terminazione TLS e il riavvio dei servizi in caso di errore.

L'URL pubblico dell'istanza di produzione è fornito da Railway (ad esempio `https://xyz.railway.app/`) e costituisce il punto di accesso esterno al sistema per gli utenti finali.

4 Tecnologie utilizzate

In questa sezione vengono elencate le tecnologie adottate per lo sviluppo del sistema *SmartOrder*. Per ciascuna tecnologia viene indicata la versione utilizzata, la motivazione della scelta e il contesto d'impiego nel progetto.

4.1 Linguaggi di programmazione

4.1.1 TypeScript

- **Versione:** 5
- **Documentazione ufficiale:** TypeScript Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend
- **Contesto di utilizzo:** linguaggio principale per l'interfaccia utente dell'applicativo Next.js, sia per i componenti React che per la gestione dello stato lato client.
- **Vantaggi:** tipizzazione statica che riduce errori a runtime, ottimo supporto IDE, compatibilità nativa con React e Next.js.
- **Svantaggi:** overhead di compilazione rispetto a JavaScript puro.

4.1.2 Python

- **Versione:** 3.11
- **Documentazione ufficiale:** Python 3 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** implementazione della pipeline di elaborazione degli ordini tramite intelligenza artificiale e gestione delle API REST tramite FastAPI.
- **Vantaggi:** ecosistema maturo in ambito NLP/ML, compatibilità nativa con le librerie OpenAI, sintassi concisa, supporto asincrono con Uvicorn/ASGI.
- **Svantaggi:** prestazioni inferiori a linguaggi compilati; mitigato dall'uso di Uvicorn (ASGI asincrono).

4.1.3 SQL

- **Documentazione ufficiale:** ISO/IEC 9075:2023 (SQL) (Ultima modifica 2026-03-26)
- **Ambito:** Database
- **Contesto di utilizzo:** interrogazioni sul catalogo prodotti, gestione ordini, sessioni e carrelli.

4.2 Framework e librerie

4.2.1 Next.js

- **Versione:** 15.5
- **Documentazione ufficiale:** Next.js Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Frontend full-stack
- **Contesto di utilizzo:** framework principale per il frontend. Gestisce il rendering dell'interfaccia utente (React) e la logica lato client.
- **Vantaggi:** App Router con Server Components, SSR e SSG, integrazione ottimale con React.
- **Svantaggi:** curva di apprendimento per le convenzioni App Router; vincolo a React come libreria UI.

4.2.2 React

- **Versione:** 19
- **Documentazione ufficiale:** React Documentation (Ultima modifica 2026-03-26)
- **Ambito:** UI
- **Contesto di utilizzo:** costruzione dell'interfaccia utente tramite componenti dichiarativi, gestione dello stato con Context API e Hooks.
- **Vantaggi:** ecosistema vastissimo, modello a componenti riutilizzabili, Virtual DOM per aggiornamenti efficienti.

4.2.3 Tailwind CSS

- **Versione:** 4.1
- **Documentazione ufficiale:** Tailwind CSS Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Styling
- **Contesto di utilizzo:** stilizzazione dei componenti React con approccio utility-first.
- **Vantaggi:** prototipazione rapida, design responsive$_G$ integrato, nessun CSS custom da mantenere.
- **Svantaggi:** classi verbose nel markup; mitigato dalla composizione in componenti React.

4.2.4 FastAPI

- **Versione:** 0.115
- **Documentazione ufficiale:** FastAPI Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** espone gli endpoint `/chat` e `/transcribe` per l'elaborazione AI, gestione degli ordini e delle sessioni. Funziona come backend principale dell'applicazione.
- **Vantaggi:** validazione automatica dei dati tramite Pydantic$_G$, documentazione OpenAPI auto-generata, supporto nativo async/await.
- **Svantaggi:** richiede un processo separato (Uvicorn) rispetto al runtime Node.js.

4.2.5 Pydantic

- **Versione:** 2.9
- **Documentazione ufficiale:** Pydantic Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** definizione dei modelli di request/response per le API FastAPI, con validazione automatica dei tipi.

4.2.6 OpenAI SDK

- **Versione:** 1.51
- **Documentazione ufficiale:** OpenAI API Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** SDK$_G$ ufficiale per l'integrazione con GPT-4o, GPT-4o-mini e Whisper.

4.3 Strumenti di persistenza

4.3.1 PostgreSQL

- **Versione:** 16
- **Documentazione ufficiale:** PostgreSQL 16 Documentation (Ultima modifica 2026-03-26)

- **Ambito:** Database
- **Contesto di utilizzo:** unica base dati del sistema, accessibile dal modulo AI tramite Python (`psycopg2`). Tutte le operazioni sul catalogo prodotti, ordini, sessioni e carrelli avvengono tramite FastAPI.
- **Vantaggi:** robustezza, supporto a query complesse e full-text search, estensibilità (pgvector per Vector Search$_G$).

4.3.2 psycopg2

- **Documentazione ufficiale:** psycopg2 Documentation (Ultima modifica 2026-03-26)
- **Ambito:** Modulo AI / Backend
- **Contesto di utilizzo:** driver Python per PostgreSQL con connection pooling$_G$ (`SimpleConnectionPool`).

4.4 Intelligenza artificiale

4.4.1 GPT-4o

- **Versione:** 2024-08-06
- **Documentazione ufficiale:** OpenAI GPT-4o (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello linguistico principale per la comprensione degli ordini, la classificazione degli intenti e le decisioni di business. Utilizzato con temperature=0.1 per risposte deterministiche.

4.4.2 GPT-4o-mini

- **Documentazione ufficiale:** OpenAI GPT-4o-mini (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello leggero per l'estrazione rapida dei parametri di ricerca e la classificazione semantica degli intenti utente.

4.4.3 Whisper

- **Versione:** 1
- **Documentazione ufficiale:** OpenAI Speech-to-Text (Whisper) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** trascrizione di messaggi audio in testo, con prompt specifico per il dominio Ergon e lingua forzata a italiano.

4.5 Infrastruttura e DevOps

4.5.1 Docker

- **Versione:** $\geq$ 24.0
- **Documentazione ufficiale:** Docker Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** containerizzazione dell'intero sistema per garantire portabilità e riproducibilità degli ambienti. Docker Compose ($\geq$ 2.20) orchestra l'avvio dei container.

4.5.2 Node.js

- **Versione:** 20
- **Documentazione ufficiale:** Node.js 20 Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** runtime JavaScript per l'esecuzione di Next.js in sviluppo e produzione.

4.5.3 Uvicorn

- **Versione:** 0.32
- **Documentazione ufficiale:** Uvicorn Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** server ASGI$_G$ per l'esecuzione di FastAPI, con supporto a richieste asincrone.

4.5.4 GitHub Actions

- **Documentazione ufficiale:** GitHub Actions Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** pipeline di integrazione continua per quality checks (ESLint, build verification).

4.5.5 ESLint

- **Versione:** 9
- **Documentazione ufficiale:** ESLint Documentation (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** linter$_G$ statico per il codice TypeScript/JavaScript.